Flipkart Typeahead Search Architecture

In-Memory FSTs, Two-Tier Candidate Retrieval & Real-Time Session ML

E-COMMERCE SEARCH

TYPEAHEAD AT SCALE

1. The Keystroke Latency & Throughput Ceiling

At Flipkart, typeahead search auto-suggest is the primary front door for over 150 million Indian e-commerce shoppers.

The Operational Challenge

- **Interactive Keystroke SLA:** Typists enter 4 to 6 characters per second. The entire backend search loop must evaluate prefixes, personal preferences, and commercial intent within **under 15 milliseconds**.
- **Festive Flash Scale:** During *The Big Billion Days (BBD)*, typeahead queries surge beyond **250,000 QPS** across the search cluster.
- **Intent Ambiguity:** Static alphabetical sorting yields poor click-through rates. Typing `"sa"` must dynamically distinguish between *Samsung TVs*, *Sarees*, and *Sanitizers* based on user context.

2. In-Memory Finite State Transducers (FST)

Traditional Prefix Tries suffer from massive memory bloat (pointers per node) and JVM GC pauses. Flipkart utilizes **Finite State Transducers (FSTs)**:

```
          Standard Prefix Trie                Finite State Transducer (FST)
    • Shares common prefixes only       • Directed Acyclic Word Graph (DAWG)
    • Heavy node pointer trees          • Shares BOTH common prefixes & suffixes
    • 120GB+ Heap Memory                • Under 4GB Off-Heap Contiguous Bytes
    • High Garbage Collection overhead  • Zero GC • Sub-3ms traversal
```

3. Two-Tier Retrieval & Ranking Architecture

```
[ User Keystroke: "kur" + Live Session Profile ]
                      | gRPC Query (<15ms SLA)
                      ▼
┌──────────────────────────────────────────────────────────────┐
│              L1 CANDIDATE RETRIEVAL TIER (Sub-3ms)           │
│ In-Memory FST Prefix Traversal • Levenshtein Distance Typo Correction  │
│ Returns top 100 candidate query strings + baseline historical CTR      │
└──────────────────────────────────┬───────────────────────────┘
                                   | 100 Candidates
                                   ▼
┌──────────────────────────────────┴───────────────────────────┐
│              L2 REAL-TIME ML RE-RANKING TIER (Sub-8ms)        │
│ Real-Time User Session Context (Category affinity, recent clicks)      │
│ LightGBM / LambdaMART ML Model ranks candidates by conversion intent   │
└──────────────────────────────────┬───────────────────────────┘
                                   | Top 10 Ranked Suggestions
                                   ▼
[ Interactive UI Dropdown: "Kurti Sets for Women", "Kurtas with Palazzo" ]
```

4. Real-Time In-Session Personalization

Flipkart blends real-time session signals into suggestion ranking rather than relying solely on global static counts:

- **Session Feature Cache:** Redis and Aerospike clusters maintain short-lived (15-minute) vectors of items viewed, categories browsed, and active cart items per user.
- **Intent-Biased Scoring:** If a shopper recently browsed ethnic fashion, typing `"si"` elevates `"silk sarees"` to the top position, while a shopper viewing phones sees `"sim card ejector"`.

5. Levenshtein Automata & Transliteration Tolerance

Over 20% of mobile queries contain misspellings or phonetic variations:

Levenshtein Automata (D=1, 2)
Traverses the FST while allowing character insertion, deletion, and substitution transitions. Catches typos like `"iphne"` ⟶ `"iphone"` in under 2ms.

Hinglish Metaphone Indexing
Maps phonetic Indian transliterations (e.g. `"joota"` ⟶ `"shoes"`, `"chappal"` ⟶ `"slippers"`) to canonical product categories.

6. Architectural Comparison: Standard Trie vs. Flipkart FST

Architectural Vector	Standard Prefix Trie	Flipkart Hyper-Personalized FST
Memory Footprint	120 GB (Heavy pointer graph)	**< 4 GB (Compressed contiguous bytes)**
Ranking Paradigm	Static popularity count	**Two-tier: L1 Retrieval + L2 Real-time ML**
Spelling Correction	Separate secondary lookup pass	Integrated Levenshtein state automata
p99 Response Time	35ms – 100ms (GC pauses)	**Sub-15ms p99 at 250,000 QPS**

7. Edge Head Caching & Graceful Degradation

To survive 250k QPS surges during festive sales without server brownouts:

- **Edge Head Caching:** The top 10,000 single and two-character prefixes (e.g., `"m"`, `"mo"`, `"s"`, `"sh"`) represent 40% of all queries. These are pre-rendered and cached directly in edge CDN nodes and client mobile memory.
- **Dynamic Load Shedding:** If backend ML scoring CPU exceeds 85%, the system sheds the L2 re-ranking pass and serves pure L1 FST popularity results, guaranteeing sub-15ms uptime under extreme duress.

8. Key Architectural Takeaways for Typeahead Systems

Core Principles Distilled from Flipkart's Typeahead Architecture

- **Replace Tries with FSTs for Extreme Compression:** Sharing both prefixes and suffixes in a Directed Acyclic Word Graph slashes RAM usage by 90% and keeps traversals contiguous and CPU cache-friendly.
- **Separate Retrieval from Personalization:** High-speed L1 candidate generation (sub-3ms) coupled with machine learning L2 re-ranking (sub-8ms) satisfies strict interactive keystroke SLAs.
- **Incorporate Real-Time In-Session Intent:** A user's immediate browsing history inside the current session is 10x more predictive of purchase intent than their long-term historical profile.
- **Cache Head Prefixes at the Network Edge:** Pre-computing common prefixes absorbs nearly half of all keystroke traffic before it ever touches core search infrastructure.